



What Is an AI Agent?

An AI Agent is an autonomous entity (software or system) that:

- Perceives its environment (through sensors or inputs),
- Thinks or reasons about what it perceives,
- Acts upon the environment (through actuators or outputs),
- And does this continuously to achieve specific goals.

👉 In simple words:

An AI agent is something that senses, thinks, and acts intelligently to achieve a goal.



Structure of an AI Agent

An AI Agent typically follows this loop (called the *Perception-Action Cycle*):

[Environment]



[Sensors] → [Agent] → [Actuators]



[Reasoning/Decision Making]

1. Sensors

Used to perceive or gather data from the environment.

- Examples:
 - In a self-driving car → Cameras, LIDAR, GPS
 - In a chatbot → Text input from the user
 - In a game bot → Game state or screen pixels

2. Agent (Brain/Decision Engine)

This is the core part of the AI system. It decides what to do next based on inputs. It includes:

- Knowledge base — what it already knows
- Reasoning/Inference engine — how it makes decisions
- Learning algorithms — how it improves with experience

3. Actuators

Used to act upon the environment.

- Examples:
 - Self-driving car → Steering, acceleration, brakes
 - Chatbot → Sending text replies
 - Robot → Moving arms or speaking

Types of AI Agents

AI Agents are categorized based on how intelligent or capable they are in reasoning and learning:

1. Simple Reflex Agents

- Act only based on current perception.
- No memory or learning.
- Follow “If condition → Then action” rules.
- Example:
A vacuum cleaner that turns when it hits a wall.

 Formula:

Action = f(Perception)

2. Model-Based Reflex Agents

- Keep track of the world using an internal model.
- Can handle partially observable environments (where not everything is visible).
- Example:
A self-driving car maintaining a map of surroundings.

 Formula:

Action = f(Perception, Internal Model)

3. Goal-Based Agents

- Have specific goals and choose actions that lead toward those goals.
- Require planning and search algorithms (like A*, BFS, DFS).

- **Example:**
A path-finding robot trying to reach a destination.

 **Formula:**
 $\text{Action} = f(\text{Perception}, \text{Goal})$

4. Utility-Based Agents

- Try to maximize a utility (happiness or performance measure).
- Handle trade-offs and uncertainty better than goal-based agents.
- **Example:**
A stock trading AI maximizing profit while minimizing risk.

 **Formula:**
 $\text{Action} = f(\text{Perception}, \text{Utility Function})$

5. Learning Agents

- Can learn from experience to improve future performance.
- Have four main components:
 1. Learning Element – Improves agent's behavior using feedback.
 2. Performance Element – Selects actions.
 3. Critic – Evaluates performance and provides feedback.
 4. Problem Generator – Suggests new exploratory actions.
- **Example:**
ChatGPT, recommendation systems, self-learning robots.

 **Formula:**
 $\text{Action} = f(\text{Perception}, \text{Learned Knowledge})$

Types of Environments for Agents

An agent's environment affects how it behaves:

Environment Type	Description	Example
Fully Observable	Agent can see the entire environment	Chess
Partially Observable	Agent can only see part of environment	Driving in fog

Environment Type	Description	Example
Deterministic	Next state is predictable	Puzzle
Stochastic	Randomness involved	Poker
Static	Environment doesn't change	Crossword
Dynamic	Environment changes over time	Traffic
Discrete	Finite states/actions	Turn-based games
Continuous	Infinite states/actions	Real-world driving



Examples of Real-World AI Agents

Domain	Example	Type of Agent
Virtual Assistants	Siri, Alexa	Goal-based + Learning
Self-driving Cars	Tesla Autopilot	Model-based + Utility-based
Game AI	Chess engines	Goal-based
Chatbots	ChatGPT	Learning + Utility-based
Recommender Systems	Netflix, YouTube	Learning-based
Trading Bots	Algorithmic trading systems	Utility-based



Key Features of AI Agents

Feature	Description
Autonomy	Operates without human intervention
Reactivity	Responds to environmental changes
Proactiveness	Takes initiative to achieve goals
Social Ability	Interacts with humans or other agents
Adaptability	Learns and improves over time



Architecture of an AI Agent

There are two common architectures:

1. Reactive Architecture

- Based purely on current inputs.
- Fast but limited intelligence.
- Used in simple robots and embedded systems.

2. Deliberative Architecture

- Uses knowledge, goals, and reasoning.
- Capable of planning, prediction, and learning.
- Used in advanced AI like autonomous vehicles and assistants.



Modern Use in Generative & Autonomous AI

Today's AI agents (like ChatGPT-based assistants, AutoGPT, and LangChain agents) use LLMs (Large Language Models) to:

- Understand complex tasks,
- Plan multi-step goals,
- Interact with APIs and tools,
- Learn from feedback and context.

For example:

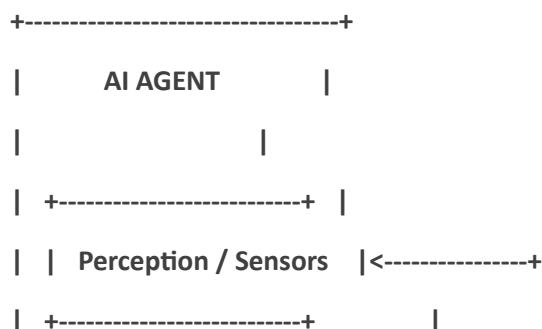


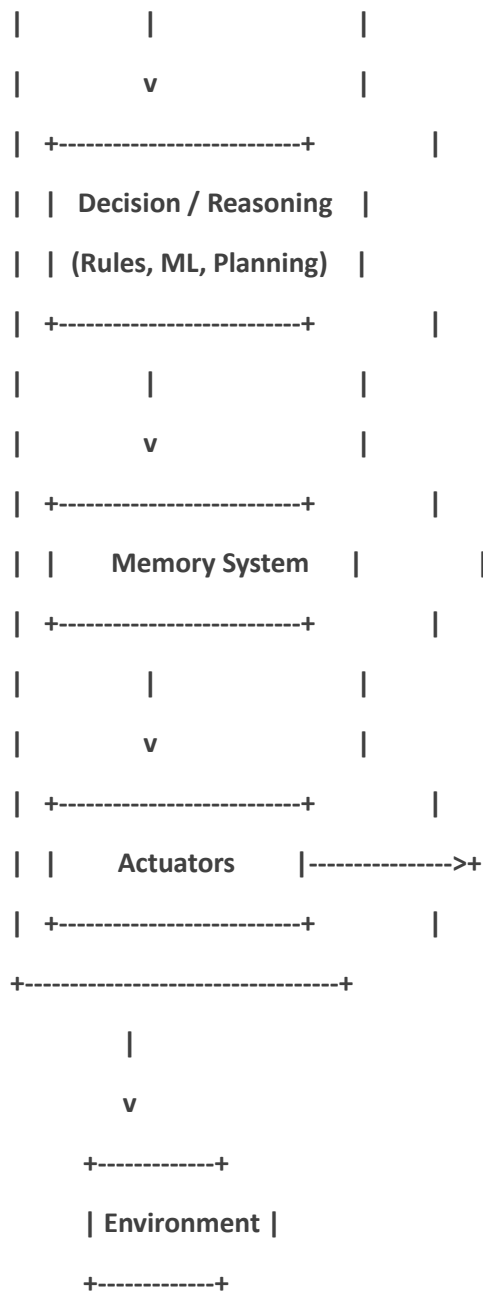
AutoGPT = A self-guided AI agent that uses GPT + memory + goals + environment interaction to execute complex tasks automatically.



1. Diagram: Structure of an AI Agent

Here's a conceptual diagram that shows the working flow of an AI Agent:





Explanation of Flow:

1. The agent perceives the environment through sensors.
2. It stores information (memory) and reasons or plans based on it.
3. It decides what to do next using its knowledge or goals.
4. It acts on the environment through actuators.
5. The environment changes, and the cycle repeats continuously.



2. Python Example — A Simple AI Agent

Let's create a simple goal-based reflex agent that tries to move from a start to a goal in a grid world.



Task:

- The agent lives in a 5x5 grid.
 - Goal = reach cell (4,4) starting from (0,0).
 - Obstacles block some cells.
 - The agent can move right or down if the path is free.
-



Python Code Example

Simple AI Agent Example - Goal Based Agent

```
class Environment:
```

```
    def __init__(self, size, obstacles, goal):
```

```
        self.size = size
```

```
        self.obstacles = obstacles
```

```
        self.goal = goal
```

```
    def is_free(self, position):
```

```
        x, y = position
```

```
        if (x, y) in self.obstacles or x >= self.size or y >= self.size:
```

```
            return False
```

```
        return True
```

```
class Agent:
```

```
    def __init__(self, environment):
```

```
        self.env = environment
```

```
        self.position = (0, 0) # Start position
```

```
self.path_memory = [] # Stores visited positions
```

```
def perceive(self):
```

```
    # Sense environment: current position and surroundings
```

```
    x, y = self.position
```

```
    surroundings = {
```

```
        "right": (x, y + 1),
```

```
        "down": (x + 1, y)
```

```
    }
```

```
    return surroundings
```

```
def decide(self, surroundings):
```

```
    # Goal-based: move towards goal if possible
```

```
    for move, pos in surroundings.items():
```

```
        if self.env.is_free(pos) and pos not in self.path_memory:
```

```
            return pos
```

```
    return None # No valid move found
```

```
def act(self, new_position):
```

```
    # Move and update memory
```

```
    if new_position:
```

```
        self.path_memory.append(new_position)
```

```
        self.position = new_position
```

```
        print(f"Moved to {self.position}")
```

```
    else:
```

```
        print("No valid moves available!")
```

```
def reached_goal(self):
```

```
    return self.position == self.env.goal
```



```
# Define environment

obstacles = {(1,1), (2,2), (3,1)}

env = Environment(size=5, obstacles=obstacles, goal=(4,4))


# Create agent

agent = Agent(env)


# Run the agent until it reaches the goal

while not agent.reached_goal():

    surroundings = agent.perceive()

    next_move = agent.decide(surroundings)

    agent.act(next_move)


print("🎯 Goal Reached!")
```

⚙️ Explanation of Code

- Environment defines:
 - Grid size, obstacles, and goal.
 - Agent:
 - Perceives surroundings (perceive()).
 - Decides where to move next based on what's free (decide()).
 - Acts by moving and recording visited positions (act()).
 - The memory here (path_memory) helps the agent avoid revisiting places.
-

3. What Is Memory in an AI Agent?

Memory is one of the most crucial parts of intelligent agents. It allows the agent to remember past experiences, learn from them, and make better future decisions.

Definition

Memory in an AI agent is a storage mechanism that holds information about past perceptions, actions, outcomes, and internal states.

It allows the agent to use *experience* for decision-making and learning.



Types of Memory in AI Agents

There are multiple ways to classify memory, but the most important are these:

1. Short-Term (Working) Memory

- Stores recent perceptions and actions.
- Used for immediate decision-making.
- Gets updated frequently (like human working memory).
- Example:
 - A chatbot remembers the last few sentences of a conversation.
 - A self-driving car remembers nearby vehicles' positions.

 Purpose: Context awareness.

2. Long-Term Memory

- Stores knowledge or experiences permanently.
- Used for learning, planning, and reasoning.
- Example:
 - Past chat histories used to fine-tune responses.
 - Historical driving data used to improve navigation.

 Purpose: Learning and improving performance over time.

3. Episodic Memory

- Stores specific events or experiences (what happened, when, and where).
- Helps the agent recall past situations to act better.
- Example:

- An AI assistant remembering when it last scheduled your meeting.

🧩 Purpose: Experience-based reasoning.

4. Semantic Memory

- Stores factual knowledge — rules, meanings, and relationships.
- Example:
 - Knowing “Paris is the capital of France” or “Traffic lights: Red means stop”.

🧩 Purpose: Understanding and reasoning logically.

5. Procedural Memory

- Stores skills and methods learned over time.
- Example:
 - A robot learning how to grasp objects.
 - A recommendation agent learning how to rank items.

🧩 Purpose: Performing actions efficiently.

⚙️ How AI Agents Use Memory

Function	Description	Example
Context Maintenance	Remembering recent data for smooth interaction	ChatGPT remembering your previous messages
Learning from Experience	Improving performance based on past outcomes	Reinforcement learning agent
State Tracking	Knowing where it currently is in a process	Game AI tracking player's position
Goal Progress	Keeping track of progress toward a goal	Task-planning agent
Adaptation	Modifying behavior over time	Smart thermostat learning user habits

Memory in Modern AI Agents (like ChatGPT, AutoGPT, LangChain, etc.)

In modern AI frameworks, memory is handled in more advanced ways:

1. Short-Term (Context Window)

- Keeps recent interactions in the model's context (like the chat you're having now).
- Limited to a few thousand tokens.

2. Long-Term (Vector Memory / Embeddings)

- Stores user information or conversation history as vector embeddings in a database.
- Allows semantic recall (e.g., remembering similar past topics).

3. Tool and Knowledge Memory

- AI agents can access external tools (APIs, databases, files) to recall or retrieve information dynamically.

Example: Memory in an Advanced AI Agent

In a system like LangChain, an AI agent may have:

- `ConversationMemory` → remembers chat context.
- `VectorStoreMemory` → stores facts and documents in embeddings.
- `SummaryMemory` → keeps summaries of long conversations to avoid forgetting older details.

Summary Table — AI Agent Memory

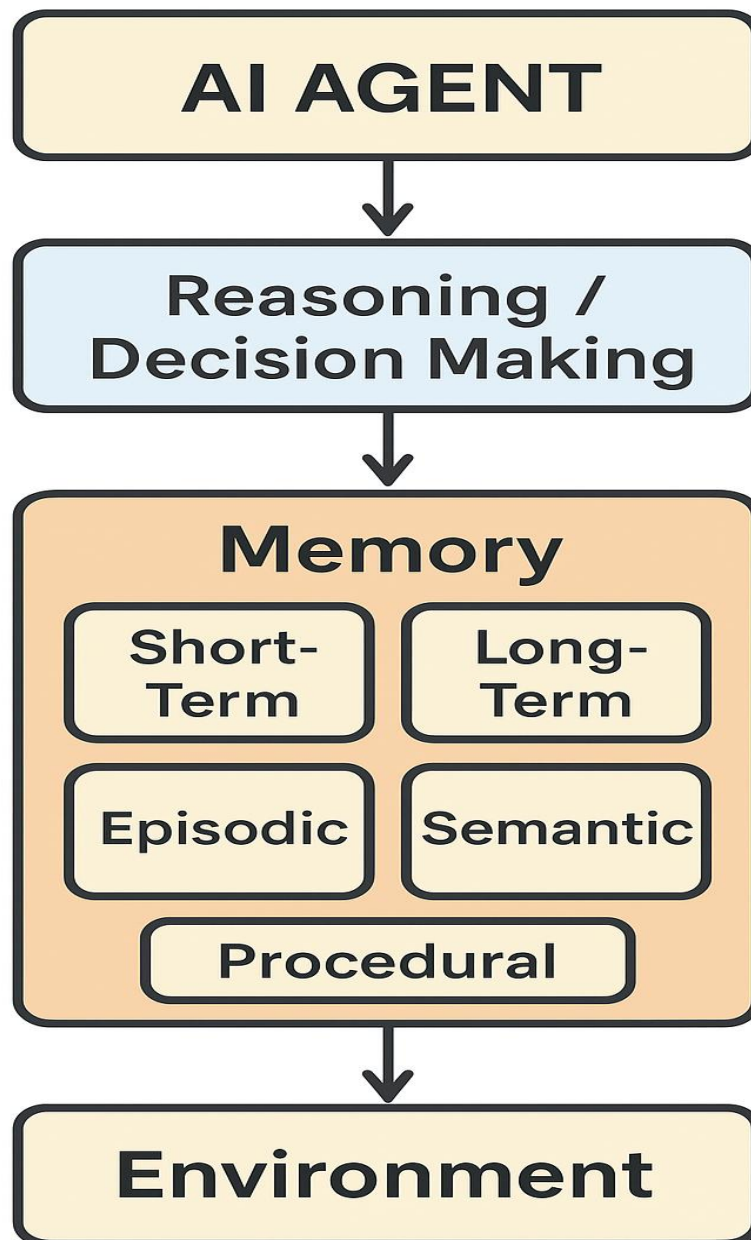
Memory Type	Description	Duration	Example
Short-Term	Recent perceptions/actions	Temporary	Current chat or scene
Long-Term	Learned knowledge	Permanent	Training data or logs
Episodic	Experiences/events	Variable	Meeting or past session
Semantic	Facts and rules	Long-term	Knowledge graphs
Procedural	Skills and actions	Long-term	Motion control or algorithms

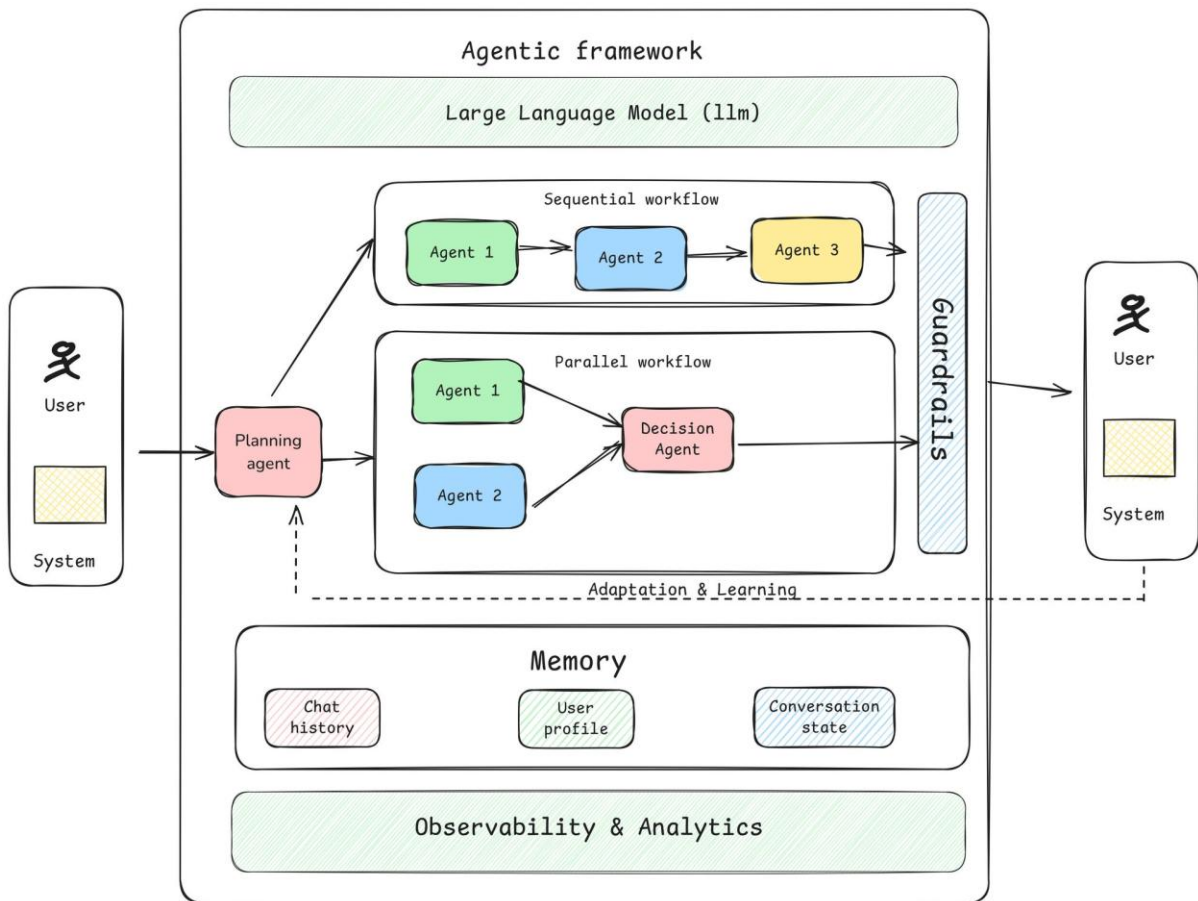
AI memory is what transforms a simple reactive system into a truly intelligent agent.
Without memory, an agent:

- Can't learn,
- Can't adapt,
- And can't reason over time.

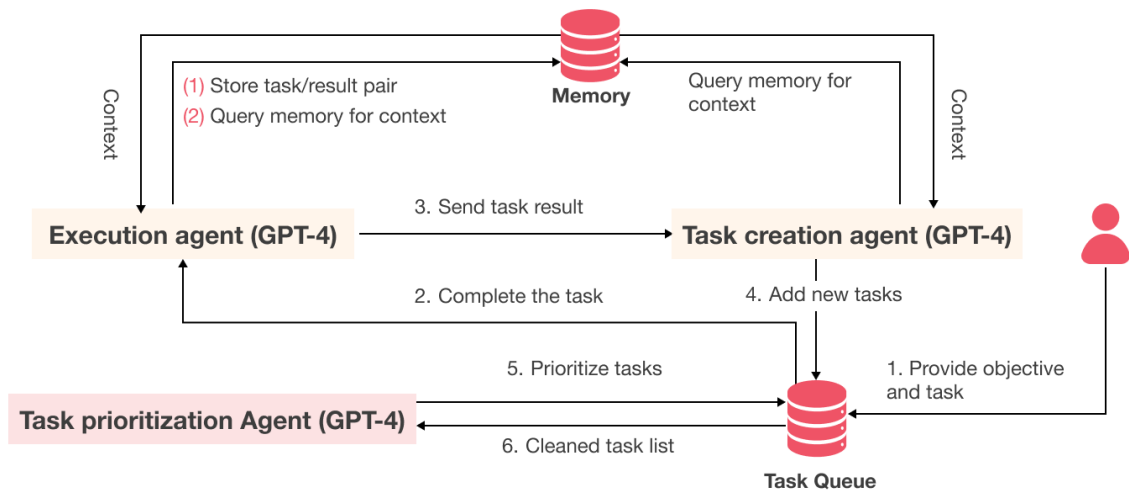
With memory, an agent can:

- ✓ Learn from experience,
- ✓ Maintain context,
- ✓ Adapt behavior dynamically, and
- ✓ Exhibit human-like intelligence.





Autonomous AI agent workflow



SIMFORM

What is Agentic AI?

Agentic AI refers to AI systems that can autonomously make decisions and take actions toward achieving specific goals, with limited or no direct human intervention.

Key aspects:

- **Autonomy:** They're able to act on their own, not just react to user prompts.
- **Goal-orientation:** They're designed to pursue objectives, plan, and execute workflows.
- **Adaptability:** They learn from experience, environment changes, feedback, and thereby improve over time.
- **Integration of action + reasoning + perception:** They don't just generate content (like a typical LLM) but also *use* it, interface with tools, and operate in an environment.

Why this distinction matters:

- Traditional AI / Generative AI often respond to a prompt ("Write me a summary") but stop there.
- Agentic AI goes further: it can plan, decide, act, and learn over multiple steps, interacting with external systems and adapting.
- In simple terms: it's "autonomous intelligent agent + goal + tools + environment" rather than "model generates output".

Historical & Conceptual Context

- The notion of "agents" in AI has existed for decades (intelligent agents, software agents, multi-agent systems).
- Agentic AI is the more recent wave, enabled by large language models (LLMs), advanced reasoning, and integration with tools/APIs.
- For example, one source notes: "Agentic automation, also known as agent-based process management, when applied to process automation."
- As the complexity of tasks and tools increased, the shift moved from narrow automation to full-workflow autonomous agents.

Why Agentic AI Matters (and What Its Benefits Are)

Benefits & transformative potential

- Completing complex, multi-step tasks: Not just “respond” but “plan → act → follow through”. [Arm+1](#)
- Seamless tool integration: Connect with APIs, databases, other systems, enabling cross-system workflows. [ibm.com+1](#)
- Personalisation & adaptation: Over time, agents remember, adapt, and refine their behaviour to users and context. [cognigy.com+1](#)
- Efficiency and scale: By automating higher-level processes, organisations can operate more efficiently and at larger scale. [sitecore.com+1](#)
- Shift from manual task-oriented work to strategic automation: Allows humans to focus on higher-value tasks while agents handle orchestration, monitoring, routine decision loops.

Summarised in one sentence: Agentic AI is the next phase of automation — not just scripting, but autonomous, intelligent, goal-driven systems.

Architecture & Core Components

Let’s break down what goes into an agentic AI system — the “how it works”.

Agentic architecture layers

Typical components include:

- Perception / Input: Collects data from environment, user, sensors, APIs. [ibm.com+1](#)
- Knowledge / Memory: Stores past interactions, learned models, long-term context.
- Planning & Reasoning: Constructs workflows, breaks down tasks into subtasks, chooses actions. [UiPath](#)
- Execution / Action: Using tools, APIs, effects on environment. [Arm+1](#)
- Feedback / Learning: Monitors outcomes, updates behaviour, refines models. [PwC+1](#)
- Orchestration / Multi-agent coordination (for more complex systems): Many agents working together, communicating, maybe supervised by a higher-level “conductor” agent. [ibm.com+1](#)

Key technical capabilities

- Goal decomposition: Agents break a large objective into smaller tasks. [UiPath](#)

- **Tool Use & API integration:** Agents call external services, read/write data, trigger actions. [Amazon Web Services, Inc.](#)
- **Memory / State:** Short-term context (recent steps) + long-term memory of previous tasks, preferences, domain knowledge. [Wikipedia](#)
- **Learning & Adaptation:** Continuous improvement via feedback loops, possibly reinforcement or tracked metrics. [arXiv+1](#)
- **Multi-agent systems:** In advanced settings, multiple specialized agents collaborate, maybe in a hierarchy or peer network. [Reddit+1](#)

Levels/Stages of Agentic AI (Maturity)

To understand where we currently are and what's ahead, here are common maturity levels (derived from practitioner commentary). From simpler to more advanced:

Level	Description
Level 1: Rule-based automation	Simple "if-then" logic, very limited autonomy.
Level 2: Assisted/Co-pilot style	Models help humans, maybe route tasks, classify, but don't independently manage workflows.
Level 3: Tool-using autonomous agents	Agent uses LLM+tools, context tracking, executes multi-step tasks. ("Current frontier") Reddit
Level 4: Multi-agent systems & self-improvement	Teams of agents coordinate; system can monitor/adjust itself.
Level 5: Fully autonomous general agentic systems	Hypothetical future where agents can handle broad domains, learn new domains autonomously.

This ladder helps you see how "agentic AI" is gradually moving from impressive demos to more autonomous, robust systems.

Use Cases & Applications

Agentic AI is being applied (or piloted) across many domains. Here are prominent examples:

- **Enterprise operations / workflow automation:** Automating entire business processes (customer service, supply chain, HR) rather than just single tasks. [Salesforce+1](#)

- IT and software modernisation: For example, migrating legacy systems (.NET → Linux) using agentic AI to analyse, decompose, transform code. [Amazon Web Services, Inc.](#)
- Customer support & contact centres: Agents that understand context, act autonomously across multiple systems, personalise experiences. [cognigy.com](#)
- Scientific research & discovery: Agents that explore datasets, propose experiments, integrate across tools and methods. (emerging)
- Web / browser automation: Agentic browsers or agents that act rather than just respond (e.g., reading websites, performing tasks). [Wikipedia+1](#)

Challenges, Risks & Considerations

Agentic AI offers huge promise, but comes with significant hurdles:

- Data quality & garbage in/agent issues: If data is poor, the autonomous decisions can be flawed. [TechRadar](#)
- Over-hype & immature projects: Many initiatives labelled “agentic” may not meet the true autonomy/robustness yet. [Reuters](#)
- Ethics, accountability & liability: When an agent acts autonomously, who is responsible for mistakes? The legal/ethical frameworks aren’t fully settled. [arXiv+1](#)
- Control & oversight: Even autonomous agents need oversight, audit trails, human-in-the-loop checkpoints. [The Economic Times](#)
- Complexity & cost: Setting up robust agentic systems (with memory, tool integration, orchestration) is complex and resource-intensive.
- Security risks / mis-use: Autonomous agents could be manipulated, behave unpredictably, or be used maliciously (prompt injection, unintended actions). > “One biggest challenge I been seeing is prompt injection attacks...” [Reddit](#)

How to Build / Implement Agentic AI (High-Level Steps)

If you were to build an agentic AI system (for example in your project or at work), here is a high-level roadmap:

1. Define goal(s) clearly: What is the agent expected to achieve? What is “done”?
2. Map the environment / tools / actions: What inputs does the agent have? What tools/APIs can it call? What domain knowledge is required?

3. **Select underlying model + reasoning layer:** Likely an LLM (or several) for reasoning, planning, language, plus modules for tool integration.
4. **Design memory/state system:** Short-term context (for a given session), long-term memory of previous tasks/users, knowledge base, learned behaviours.
5. **Implement planning & control logic:** How to decompose tasks, schedule subtasks, handle failures, loop until goal completion.
6. **Integrate tools/APIs/actions:** Enable the agent to act – call services, query databases, trigger workflows, act in the real world.
7. **Feedback/learning loop:** Track performance, outcomes, errors; update policies or models accordingly.
8. **Monitoring, oversight & governance:** Logging, audit trails, human-in-the-loop fallback, security controls.
9. **Scale & orchestration:** If multiple agents needed, design coordination, orchestration layer, communication protocols.
10. **Evaluate & iterate:** Measure KPIs (time saved, accuracy, success rate), iterate improvements.

Future Trends & What's Next

- **Model-native agentic AI:** A shift from external orchestration pipelines to “agentic capabilities internalised inside the model” (planning, tool-use, memory) as one survey puts it. [arXiv](#)
- **More multi-agent ecosystems:** Agents specialised in domains cooperating, maybe hierarchies of agents, shared memory, emergent coordination.
- **Better memory & lifelong learning:** Agents that don't just complete tasks in one session but accumulate knowledge, adapt across sessions, improve behaviour over time.
- **Wider enterprise adoption + governance frameworks:** As maturity increases, more companies will deploy agentic AI in production; parallel growth of governance, regulation.
- **New interfaces & human-agent collaboration paradigms:** Agents may become more like teammates, with humans supervising/training them rather than dictating every step.
- **Ethical/legal frameworks catch-up:** As agentic systems act more autonomously, issues like accountability, IP, bias, transparency become even more critical. [arXiv](#)

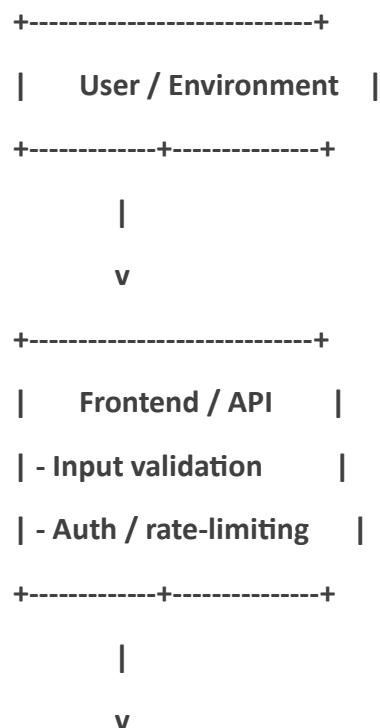
To wrap up:

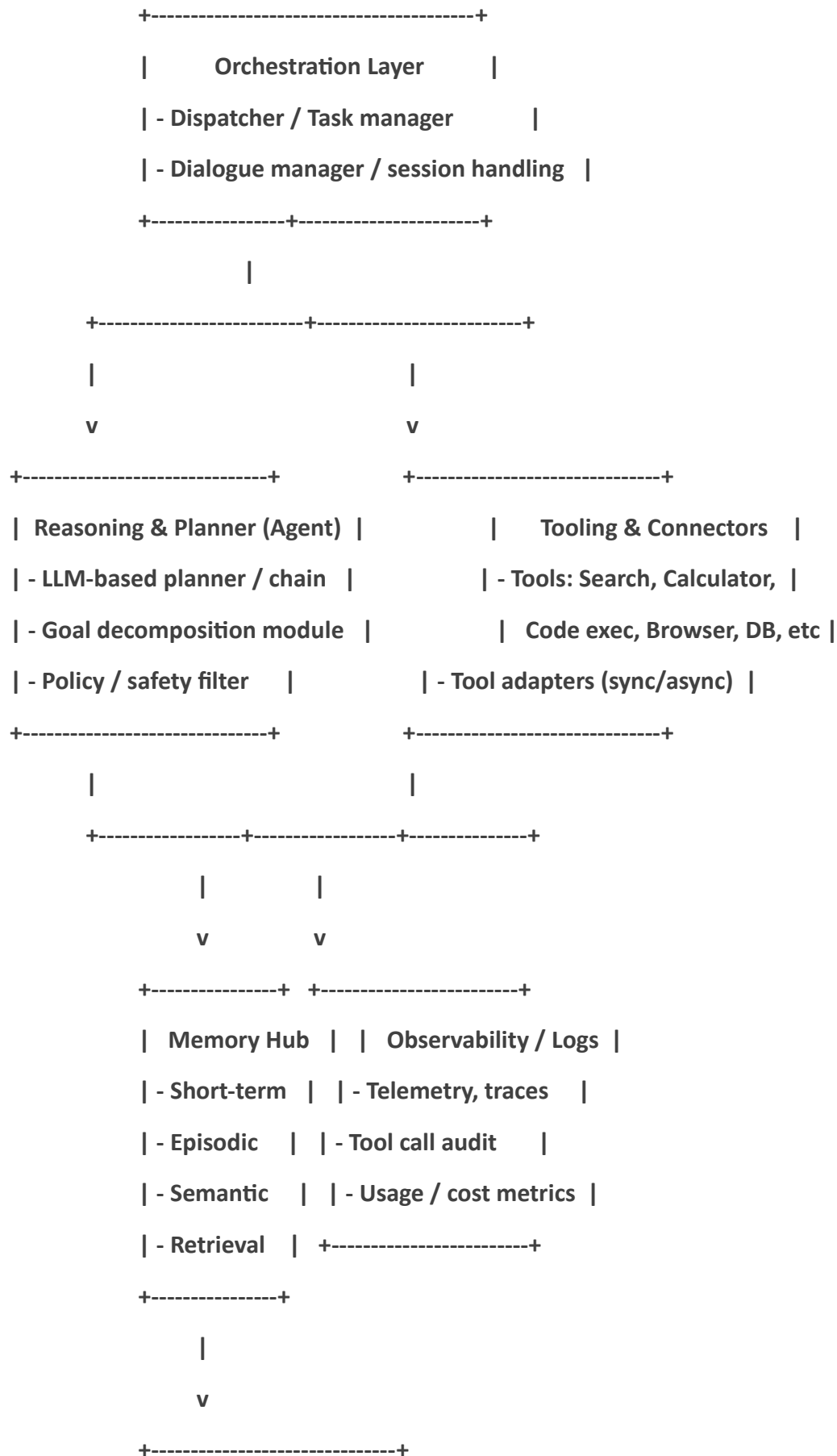
- Agentic AI = autonomous, goal-oriented, adaptive AI systems (agents) that perceive, reason, act, learn.
- It extends beyond classical AI or generative models by adding *agency*, multi-step workflows, tool integration, memory, environment interaction.
- It's influential because it enables new levels of automation, efficiency, and intelligent behaviour.
- Building it requires architecture that supports planning, memory, action, learning, orchestration.
- It comes with real challenges (data, reliability, ethics, cost).
- The field is rapidly evolving; we're moving toward more capable, autonomous, multi-agent systems.

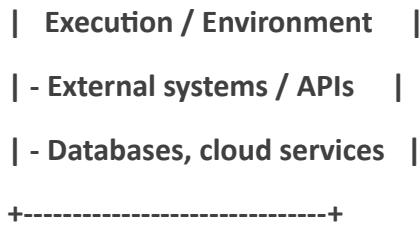
Real-world Agentic AI Architecture — Diagram, Walkthrough & Python Prototype

This document contains a labeled architecture diagram (ASCII + explanation), a deep walkthrough of each component (with memory types), safety/guardrails, and a runnable Python prototype demonstrating a minimal agentic workflow: goal decomposition → planning → tool calls → memory updates → iterative refinement.

1) High-level architecture (ASCII diagram)







2) Component-by-component explanation

Frontend / API

- Accepts user goals (short text, structured request, or a larger objective).
- Handles input sanitization, user identity, and session state.

Orchestration Layer

- Keeps track of sessions / dialogues and ensures tasks are executed, retried, and logged.
- Decides which agent or sub-agent should run.

Reasoning & Planner (Agent core)

- Goal decomposition: large goals are split into ordered subtasks. Could be hierarchical (HTN-style) or goal-graph.
- Planner: builds a sequence of steps (plan) that call tools / memory and perform checks.
- Policy / Safety filter: post-processes candidate actions to block unsafe operations (e.g., disallowed API calls, PII exfiltration).

Tooling & Connectors

- Tools are small, well-defined functions with typed inputs/outputs (e.g., `search(query) -> results`, `calc(expr) -> number`).
- The agent calls tools through adapters which enforce timeouts, retries, and input/output sanitization.

Memory Hub

- Centralized store exposing retrieval APIs to the agent. Memory types:
 - Working memory (short-term / context): ephemeral, holds the current conversation or plan (minutes-hours).
 - Episodic memory: chronological records of past sessions or actions (useful for summarization and long-term user preferences).

- Semantic memory (knowledge): persistent facts about the user or domain (e.g., preferences, frequently used templates).
- Procedural memory: saved procedures / macros (e.g., "how to run dataset split").
- Retrieval uses embeddings + vector DB or key-value store and may include heuristics for recency and relevance.

Observability / Logs

- All tool calls and actions are logged with traces, inputs, outputs, latencies, and costs.
- Critical for debugging, compliance, and audit.

Execution Environment

- Real systems and APIs the agent manipulates: databases, web services, cloud infra, user devices, etc.
- Secure connectors and least-privilege roles required.

3) Memory for AI agents — detailed explanation

Short-term / Working Memory

- Purpose: store immediate context — user message, current plan, intermediate results.
- Characteristics: high-throughput, ephemeral, cleared when the session ends or goal completes.
- Implementation: in-process structures or fast Redis caches.

Episodic Memory

- Purpose: store event logs of interactions ("on 2025-10-27 user asked X and agent did Y").
- Characteristics: append-only, used for later summarization or to reconstruct conversation state.
- Implementation: time-series DB / append-only store + summarization routines.

Semantic Memory

- Purpose: long-lived facts and preferences (e.g., "user prefers concise answers").
- Characteristics: updated occasionally, used for personalization.
- Implementation: vector DB with embeddings + metadata store.

Procedural Memory

- Purpose: saved multi-step routines that the agent can re-run.
- Characteristics: versioned, tested.
- Implementation: code repository / workflow definitions (e.g., Airflow, Prefect flows), or serialized plan templates.

Retrieval & Relevance

- Use embedding + similarity search for natural-language recall.
 - Apply recency boosts, relevance filters, and hard constraints (e.g., never return secrets).
-

4) Safety, guardrails, and policy enforcement

- Validate tool inputs; sanitize outputs (e.g., redact PII).
 - Use a policy engine to check planned actions before execution (allow/deny list).
 - Limit privileges per-tool (principle of least privilege).
 - Human-in-the-loop for high-risk actions.
-

5) Minimal Python prototype (runnable)

This is a single-file prototype showing:

- A simple Agent class
- Goal decomposition (text → subtasks)
- Planner that sequences tool calls
- Memory implementation with short-term, episodic, and semantic layers (in-memory simulated)
- Two toy tools: search_tool (simulated) and calc_tool (real evaluation, sandboxed)

Notes before running: This prototype is educational. Do not run eval on untrusted input in production. Here we sandbox calc using ast parsing.

```
# agentic_prototype.py
```

```
import time
```

```
import uuid
```

```
import ast
```

```
import operator as op
```



```
from typing import List, Dict, Any, Tuple
```

```
# ----- Simple memory -----
```

```
class MemoryHub:
```

```
    def __init__(self):
```

```
        self.working = {}          # session_id -> list of messages
```

```
        self.episodic = []        # list of (timestamp, session_id, event)
```

```
        self.semantic = {}        # key -> value (persistent prefs/facts)
```

```
    # Working memory API
```

```
    def push_working(self, session_id: str, item: str):
```

```
        self.working.setdefault(session_id, []).append((time.time(), item))
```

```
    def get_working(self, session_id: str, last_n: int = 10):
```

```
        return [it for (_, it) in self.working.get(session_id, [])[-last_n:]]
```

```
    # Episodic memory API
```

```
    def log_episode(self, session_id: str, event: str):
```

```
        self.episodic.append((time.time(), session_id, event))
```

```
    def query_episodes(self, session_id: str = None, limit: int = 10):
```

```
        filtered = [e for e in self.episodic if (session_id is None or e[1] == session_id)]
```

```
        return filtered[-limit:]
```

```
    # Semantic memory API
```

```
    def set_fact(self, key: str, value: Any):
```

```
        self.semantic[key] = value
```

```
    def get_fact(self, key: str, default=None):
```

```
return self.semantic.get(key, default)
```

```
# ----- Tools -----
```

```
# Tool: safe calculator using ast
```

```
ALLOWED_OPERATORS = {
```

```
    ast.Add: op.add,
```

```
    ast.Sub: op.sub,
```

```
    ast.Mult: op.mul,
```

```
    ast.Div: op.truediv,
```

```
    ast.Pow: op.pow,
```

```
    ast.USub: op.neg,
```

```
}
```

```
def safe_eval_expr(expr: str):
```

```
    """Evaluate simple math expressions safely."""
```

```
    def _eval(node):
```

```
        if isinstance(node, ast.Num):
```

```
            return node.n
```

```
        if isinstance(node, ast.BinOp):
```

```
            left = _eval(node.left)
```

```
            right = _eval(node.right)
```

```
            return ALLOWED_OPERATORS[type(node.op)](left, right)
```

```
        if isinstance(node, ast.UnaryOp):
```

```
            return ALLOWED_OPERATORS[type(node.op)](_eval(node.operand))
```

```
        raise ValueError("Unsupported expression")
```

```
node = ast.parse(expr, mode='eval').body
```

```
return _eval(node)
```

Tool: simulated search

```
def search_tool(query: str) -> List[str]:
```

```
    # In real system: call search API, scrape, return results with metadata
```

```
    sample = [
```

```
        f"Result A for '{query}'", f"Result B for '{query}'",
```

```
        f"Summary: something relevant about {query}"
```

```
    ]
```

```
    time.sleep(0.2)
```

```
    return sample
```

----- Agent -----

```
class Agent:
```

```
    def __init__(self, memory: MemoryHub):
```

```
        self.memory = memory
```

```
    def decompose_goal(self, goal: str) -> List[str]:
```

```
        # Very naive decomposition: split by verbs/clauses. Replace with LLM in real system.
```

```
        # For the example, we return 2-3 steps.
```

```
        steps = []
```

```
        if 'research' in goal.lower() or 'find' in goal.lower():
```

```
            steps = [
```

```
                'search: collect top-3 sources about ' + goal,
```

```
                'summarize: create a short summary of findings',
```

```
            ]
```

```
        elif 'calculate' in goal.lower() or any(ch.isdigit() for ch in goal):
```

```
            steps = [
```

```
                'calc: evaluate expression from the goal',
```

```
            ]
```

```
        else:
```

```

steps = [
    'search: gather info about ' + goal,
    'summarize: short summary',
]
return steps

```

```

def plan_and_execute(self, session_id: str, goal: str) -> Dict[str, Any]:

```

```

    plan = self.decompose_goal(goal)

```

```

    trace = []

```

```

    self.memory.push_working(session_id, f"Goal: {goal}")

```

```

    self.memory.log_episode(session_id, f"Plan created: {plan}")

```

```

    for step in plan:

```

```

        if step.startswith('search:'):

```

```

            query = step.split(':', 1)[1].strip()

```

```

            self.memory.push_working(session_id, f"ToolCall: search({query})")

```

```

            results = search_tool(query)

```

```

            trace.append({'step': step, 'result': results})

```

```

            self.memory.log_episode(session_id, f"Search results for {query}: {results}")

```

```

        elif step.startswith('calc:'):

```

```

            expr = step.split(':', 1)[1].strip() or goal

```

```

            self.memory.push_working(session_id, f"ToolCall: calc({expr})")

```

```

            try:

```

```

                val = safe_eval_expr(expr)

```

```

            except Exception as e:

```

```

                val = f"calc-error: {e}"

```

```

            trace.append({'step': step, 'result': val})

```

```

            self.memory.log_episode(session_id, f"Calc result for {expr}: {val}")

```

```
elif step.startswith('summarize:') or step == 'summarize: short summary' or
step.startswith('summarize'):
```

```
    # create a naive summary from previous working memory
```

```
    recent = self.memory.get_working(session_id, last_n=10)
```

```
    summary = ' | '.join(recent[-6:])
```

```
    trace.append({'step': step, 'result': summary})
```

```
    self.memory.log_episode(session_id, f"Summary created: {summary}")
```

```
else:
```

```
    trace.append({'step': step, 'result': 'unknown-step-type'})
```

```
    self.memory.log_episode(session_id, f"Unknown step: {step}")
```

```
return {
```

```
    'session_id': session_id,
```

```
    'goal': goal,
```

```
    'plan': plan,
```

```
    'trace': trace,
```

```
}
```

```
# ----- Example run -----
```

```
if __name__ == '__main__':
```

```
    mem = MemoryHub()
```

```
    agent = Agent(mem)
```

```
    sess = str(uuid.uuid4())
```

```
    goal = 'Research top methods for summarization and calculate 2+2'
```

```
    out = agent.plan_and_execute(sess, goal)
```

```
    import json
```

```
    print(json.dumps(out, indent=2, ensure_ascii=False))
```

How the prototype maps to real components

- **MemoryHub** simulates short-term / episodic / semantic memory. Replace with Redis + vector DB + SQL in production.
 - `decompose_goal` is a stub: replace with an LLM call that returns structured subtasks.
 - Tools are functions with controlled interfaces. In production, they are wrapped with authentication, rate-limiting, timeouts, and input validators.
-

6) Practical tips for production

1. Design tools as narrow interfaces — small surface area reduces risk.
2. Use an explicit plan representation (JSON tasks) rather than freeform text to call tools.
3. Audit every tool call — store inputs, outputs, and a hash to detect tampering.
4. Rate-limit & quota tools for cost control.
5. Human fallback for ambiguous or high-risk actions.
6. Regularly summarize episodic memory to avoid vector DB bloat (e.g., daily or weekly condensation).